Generic Lists

Generic data types

- through generic data types the data type of members (methods/properties) can be made variable
- generic types are defined with the placeholder "T
- within the class then objects of the data type T can be created

- often used for listings, which are to exhibit the same basic behavior no matter with which data type

List<T>

- a list of objects <T> that can be accessed by index
- predefined methods for adding, removing, searching, sorting, and editing

```
List<string> stringList = new List<string>();
stringList.Add("This");
stringList.Add("is");
stringList.Add("a");
stringList.Add("list");
stringList.Add("of");
stringList.Add("strings");
foreach (string item in stringList)
{
    Console.WriteLine(item);
}

//This
//is
//a
//list
//of
//strings
```

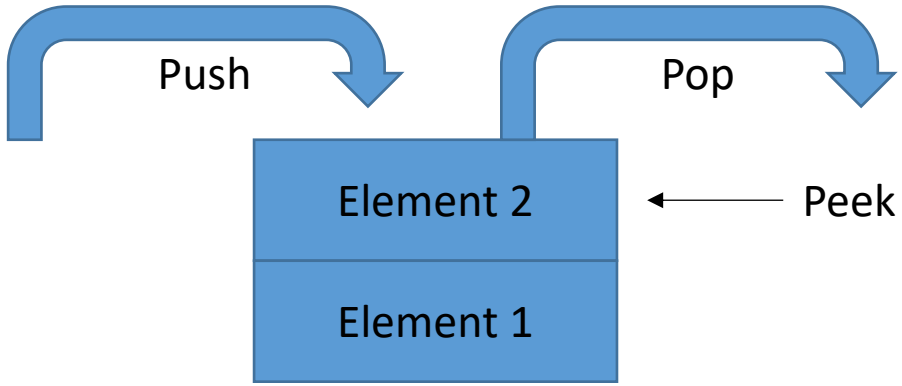
List<T> - useful functions

Name	Description
Add(T)	Adds an object to the list
Remove(T)	removes the first object T
Clear()	removes all elements from the list
Find(Predicate<T>)	returns an object corresponding to the Predicate<T>.
Contains(T)	returns whether the object T is contained in the list
Exist(Predicate<T>)	returns whether an object corresponding to the Predicate<T> exists
Sort(Comparison<T>)	sorts the list using the specified Comparison<T>.

- The more complex methods (Find(), Contains(), etc.) are found as extension methods in the namespace System.Linq

specials lists – Stack<T> (Last In First Out)

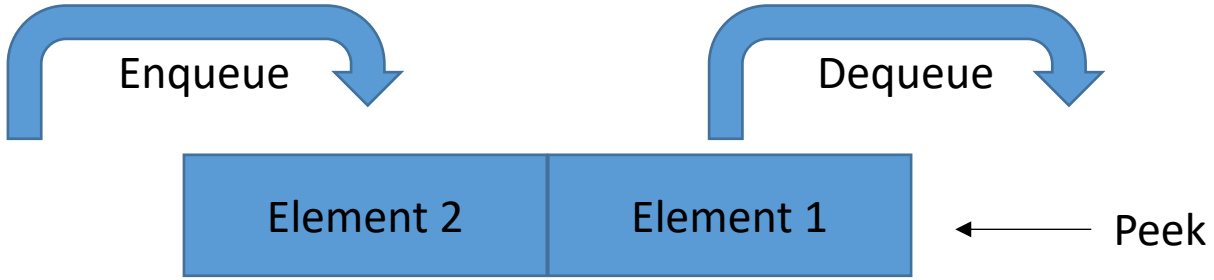
- Push(T): Add element ('put on top')
- Peek(): return top element
- Pop(): return and remove top element




```
Stack<string> stringStack = new Stack<string>();
stringStack.Push("Element 1");
stringStack.Push("Element 2");
stringStack.Peek(); //Element 2
stringStack.Pop(); //remove Element 2
```

special lists – Queue<T> (First In First Out)

- Enqueue(T): Add element ('append‘)
- Peek(): return oldest element
- Dequeue(): return and remove oldest element




```
Queue<string> stringQueue = new Queue<string>();
stringQueue.Enqueue("Element 1");
stringQueue.Enqueue("Element 2");
stringQueue.Enqueue("Element 3");
stringQueue.Peek(); //Element 1
stringQueue.Dequeue(); //remove Element 1
```

special lists – Dictionary<TKey, TValue>

- Corresponds to 'Table with 2 columns
- Assigns a Value (TValue) to each Key (TKey)
- Add(TKey, TValue): Add element
- access to the value via key specification
- ContainsKey(TKey) checks if a key is available
- ContainsValue(TValue) checks if a Value is present

```
Dictionary<int, string> stringDictionary = new Dictionary<int, string>();
stringDictionary.Add(5, "Element 1");
stringDictionary.Add(10, "Element 2");
string value10 = stringDictionary[10]; // "Element 2"
stringDictionary.ContainsKey(5); //true
stringDictionary.ContainsValue("Element 3"); //false
```